

EliteRestaurant Project - Comprehensive Technical Documentation

This report documents the complete repository structure, architecture, data model, business logic, algorithms, and execution flow of the EliteRestaurant project.

Generated: 2026-04-02 04:39

Scanned source files: 70 (52 C#, 17 XAML, 1 project files), total 11,741 lines.

1) Repository Scope and Technology Stack

- Primary application: EliteRestaurantPro, a desktop Windows app built with .NET 8 and WPF.
- Architectural pattern: MVVM (Models, ViewModels, XAML Views, command-based actions through RelayCommand).
- Persistence layer: SQLite via Entity Framework Core (Microsoft.EntityFrameworkCore.Sqlite).
- Exporting and reporting dependencies: ClosedXML for Excel and QuestPDF for in-app ticket/report PDFs.
- Database is local, created in the user profile under LocalApplicationData/EliteRestaurantPro.

Core boot files: App.xaml / App.xaml.cs, MainWindow.xaml / MainWindow.xaml.cs, MainViewModel.cs.

2) Folder Structure and Purpose

- Data/: AppDbContext and FinancialTransactionService. Handles schema evolution, seed data, and finance synchronization rules.
- Models/: Entity classes and DTO-like view models for UI projection (orders, activity timeline, reservation entry placeholder, etc.).
- ViewModels/: Feature-level business logic and state handling for dashboard, staff, menu, inventory, attendance, orders, finance, and reports.
- Views/: XAML pages and corresponding code-behind files. Includes reusable components under Views/Components.
- Resources/: Shared style dictionaries and visual resources consumed by all views.
- Utils/: Utility services (UniqueIdGenerator and ExcelExportService).

3) Startup Pipeline and Navigation Logic

- Application startup in App.OnStartup sets QuestPDF community license and calls AppDbContext.Initialize().
- MainViewModel starts at RoleSelectionViewModel and controls a single CurrentViewModel for screen routing.
- App.xaml registers DataTemplate mappings from each ViewModel type to its XAML view; no manual view switching code is required.
- AdminLoginViewModel currently uses demo authentication logic: any non-empty credentials move to admin dashboard.
- AdminBaseViewModel defines shared navigation commands to all management pages and logout.

4) Database Context, Schema Governance, and Seeding

AppDbContext is both the EF Core DbContext and migration/repair orchestrator. Instead of EF migration files, it applies runtime schema checks and ALTER/CREATE operations to keep legacy databases compatible.

- DbSets: Employees, Products, Tables, Orders, OrderItems, InventoryItems, ProductIngredients, EmployeeAttendances, Transactions.

- Schema correction methods include EnsureSchema, EnsureColumn, table creation methods, and Orders table rebuild logic when required fields are missing.
- Unique indexes are enforced for entity UniqueId values and business constraints like TableNumber and (EmployeeId, WorkDate) attendance uniqueness.
- Post-creation normalizers enforce non-empty IDs, deduplicate IDs, normalize category/subcategory naming, and assign fallback table numbers.
- Bootstrap data generation expands staff, inventory, menu catalog, shifts, table coverage, product ingredients, and historical activity for last 14 days.

5) Domain Model Inventory

- Employee: identity, role, PIN, contact, hourly rate, status, join date, notes, and per-day shift schedule.
- EmployeeAttendance: one row per employee/day with clock-in, clock-out, status, and late justification.
- Product: menu item with category, subcategory, unit price, and ingredient links.
- InventoryItem: stock entity with quantity, unit, optional expiration, notes timeline, and ingredient references.
- ProductIngredient: junction entity defining quantity of inventory consumed per product.
- Table: dining table metadata, status, seating capacity, and optional assigned server.
- OrderRecord + OrderItem: normalized order header/items with status lifecycle and table/server references.
- MoneyTransaction: ledger entry with amount, type (Revenue/Expense), category, fixed/variable flag, and justification.
- UI helper models: OrderEntry (projection), ActivityItem (dashboard timeline), ReservationEntry (present but unused).

6) Feature Modules (ViewModels) and Responsibilities

- AdminDashboardViewModel: KPI aggregation (today sales, active staff, low stock, order counts), chart generation, and drilldown launching.
- EmployeesViewModel: CRUD for employees, profile image path handling, and validation.
- MenuViewModel: menu CRUD, category/subcategory grouping logic, ingredient summaries, and grouped collections for UI.
- InventoryViewModel: inventory CRUD, stock adjustments with notes history, and low-stock awareness.
- AttendanceViewModel: clock-in/out workflow, late handling (after configured threshold), per-day attendance rows.
- TablesViewModel: table CRUD with assignment/status controls and next-table-number computation.
- AdminOrdersViewModel: order list, status transitions, completion/cancellation, table occupancy refresh, ticket preview/export.
- CreateOrderViewModel: order composition, quantity control, inventory feasibility checks, totals/taxes/service math, and draft persistence.
- MoneyViewModel: transaction entry rules, revenue/salary backfill calls, date range reporting, and excel exports.
- ReportsViewModel: employee/table/inventory/menu/day-level report aggregation with timeline grouping and export.
- DashboardDrilldownViewModel: detailed drilldown datasets and activity timeline builder.
- MainViewModel / RoleSelectionViewModel / AdminLoginViewModel: top-level routing and login entry workflow.

7) Critical Business Logic and Algorithms

- Order totals: subtotal from line items, then tax (7%) and service charge (10%) are applied in order workflows.

- **Preparation estimate:** each selected item computes prep minutes by category/subcategory, then parallelism logic caps aggregate wait time.
- **Inventory depletion algorithm:** product ingredients are multiplied by ordered quantity; order is rejected if any required stock is insufficient.
- **Table occupancy synchronization:** active order states mark tables Occupied; completion/cancellation returns them to Available unless in Maintenance.
- **Automatic revenue posting:** completed orders are converted to MoneyTransaction entries exactly once using a deterministic justification key.
- **Scheduled payroll expenses:** for each active employee and each Friday in range, an Expense/Salary transaction is created at $\text{hourlyRate} * 40$.
- **Historical activity simulation:** deterministic random seed creates attendance/orders over prior days while respecting shift patterns.
- **Unique identifier strategy:** human-readable prefixes (EMP, TBL, MEN, INV, ORD) + GUID fragment to preserve uniqueness across entities.

8) End-to-End Operational Flows

8.1 Create and Fulfill an Order

- **User** opens Create Order screen, selects table + server + menu products.
- **System** recalculates totals and estimated prep time whenever selections change.
- **On create**, inventory sufficiency is validated against ProductIngredients.
- **If valid**, inventory stocks are decremented and notes are appended for traceability.
- **Order status** enters lifecycle (Waiting -> In Kitchen -> Ready -> Completed).
- **On completion**, FinancialTransactionService posts a corresponding revenue ledger row.

8.2 Attendance and Payroll Data

- **Clock-in** only applies to current date; late threshold triggers justification workflow.
- **Clock-out** updates same attendance row for the day.
- **Money screen** invokes salary scheduler to ensure missing Friday payroll expense entries are backfilled.

8.3 Reporting and Exports

- **ReportsViewModel** computes aggregate and timeline-oriented report DTOs from transactional tables.
- **ExcelExportService** can export either a single sheet or multi-sheet workbook through ClosedXML.
- **AdminOrdersViewModel** uses QuestPDF to export printable order tickets.

9) File-by-File Source Index (Primary Worktree)

The following index lists all source files (.cs, .xaml, .csproj) excluding generated bin/obj artifacts.

```
EliteRestaurantPro/App.xaml (71 lines)
EliteRestaurantPro/App.xaml.cs (15 lines)
EliteRestaurantPro/AssemblyInfo.cs (10 lines)
EliteRestaurantPro/Data/AppDbContext.cs (1029 lines)
EliteRestaurantPro/Data/FinancialTransactionService.cs (116 lines)
EliteRestaurantPro/EliteRestaurantPro.csproj (20 lines)
EliteRestaurantPro/MainWindow.xaml (72 lines)
EliteRestaurantPro/MainWindow.xaml.cs (38 lines)
```

EliteRestaurantPro/Models/ActivityItem.cs (8 lines)
EliteRestaurantPro/Models/Employee.cs (89 lines)
EliteRestaurantPro/Models/EmployeeAttendance.cs (13 lines)
EliteRestaurantPro/Models/InventoryItem.cs (44 lines)
EliteRestaurantPro/Models/MoneyTransaction.cs (12 lines)
EliteRestaurantPro/Models/OrderEntry.cs (29 lines)
EliteRestaurantPro/Models/OrderItem.cs (11 lines)
EliteRestaurantPro/Models/OrderRecord.cs (19 lines)
EliteRestaurantPro/Models/Product.cs (12 lines)
EliteRestaurantPro/Models/ProductIngredient.cs (11 lines)
EliteRestaurantPro/Models/ReservationEntry.cs (28 lines)
EliteRestaurantPro/Models/Table.cs (13 lines)
EliteRestaurantPro/Resources/Styles.xaml (710 lines)
EliteRestaurantPro/Utils/ExcelExportService.cs (55 lines)
EliteRestaurantPro/Utils/UniqueIdGenerator.cs (10 lines)
EliteRestaurantPro/ViewModels/AdminBaseViewModel.cs (40 lines)
EliteRestaurantPro/ViewModels/AdminDashboardViewModel.cs (374 lines)
EliteRestaurantPro/ViewModels/AdminLoginViewModel.cs (61 lines)
EliteRestaurantPro/ViewModels/AdminOrdersViewModel.cs (597 lines)
EliteRestaurantPro/ViewModels/AttendanceViewModel.cs (315 lines)
EliteRestaurantPro/ViewModels/BaseViewModel.cs (20 lines)
EliteRestaurantPro/ViewModels/CreateOrderViewModel.cs (704 lines)
EliteRestaurantPro/ViewModels/DashboardDrilldownViewModel.cs (121 lines)
EliteRestaurantPro/ViewModels/EmployeesViewModel.cs (421 lines)
EliteRestaurantPro/ViewModels/InventorySelectionItemViewModel.cs (25 lines)
EliteRestaurantPro/ViewModels/InventoryViewModel.cs (363 lines)
EliteRestaurantPro/ViewModels/MainViewModel.cs (22 lines)
EliteRestaurantPro/ViewModels/MenuViewModel.cs (467 lines)
EliteRestaurantPro/ViewModels/MoneyViewModel.cs (688 lines)
EliteRestaurantPro/ViewModels/ProductSelectionItemViewModel.cs (33 lines)
EliteRestaurantPro/ViewModels/RelayCommand.cs (40 lines)
EliteRestaurantPro/ViewModels/ReportsViewModel.cs (810 lines)
EliteRestaurantPro/ViewModels/RoleSelectionViewModel.cs (17 lines)
EliteRestaurantPro/ViewModels/TablesViewModel.cs (310 lines)
EliteRestaurantPro/Views/AdminDashboardView.xaml (406 lines)
EliteRestaurantPro/Views/AdminDashboardView.xaml.cs (8 lines)
EliteRestaurantPro/Views/AdminLoginView.xaml (99 lines)
EliteRestaurantPro/Views/AdminLoginView.xaml.cs (15 lines)
EliteRestaurantPro/Views/AdminOrdersView.xaml (387 lines)
EliteRestaurantPro/Views/AdminOrdersView.xaml.cs (8 lines)
EliteRestaurantPro/Views/AttendanceView.xaml (159 lines)
EliteRestaurantPro/Views/AttendanceView.xaml.cs (11 lines)
EliteRestaurantPro/Views/Components/AdminSidebar.xaml (297 lines)

EliteRestaurantPro/Views/Components/AdminSidebar.xaml.cs (81 lines)
EliteRestaurantPro/Views/CreateOrderView.xaml (308 lines)
EliteRestaurantPro/Views/CreateOrderView.xaml.cs (11 lines)
EliteRestaurantPro/Views/DashboardDrilldownView.xaml (76 lines)
EliteRestaurantPro/Views/DashboardDrilldownView.xaml.cs (11 lines)
EliteRestaurantPro/Views/EmployeesView.xaml (399 lines)
EliteRestaurantPro/Views/EmployeesView.xaml.cs (11 lines)
EliteRestaurantPro/Views/InventoryView.xaml (242 lines)
EliteRestaurantPro/Views/InventoryView.xaml.cs (11 lines)
EliteRestaurantPro/Views/MenuView.xaml (281 lines)
EliteRestaurantPro/Views/MenuView.xaml.cs (11 lines)
EliteRestaurantPro/Views/MoneyView.xaml (279 lines)
EliteRestaurantPro/Views/MoneyView.xaml.cs (11 lines)
EliteRestaurantPro/Views/ReportsView.xaml (350 lines)
EliteRestaurantPro/Views/ReportsView.xaml.cs (11 lines)
EliteRestaurantPro/Views/RoleSelectionView.xaml (168 lines)
EliteRestaurantPro/Views/RoleSelectionView.xaml.cs (8 lines)
EliteRestaurantPro/Views/TablesView.xaml (208 lines)
EliteRestaurantPro/Views/TablesView.xaml.cs (11 lines)

10) Architectural Risks and Suggested Improvements

- Authentication is currently demo-level (non-empty credentials). Implement secure credential verification and role-based authorization.
- Runtime schema mutation in AppDbContext is practical but high-risk long-term; consider formal EF migrations for reproducibility.
- No automated tests are present. Prioritize unit tests for business rules and integration tests for order/inventory/finance flows.
- Very large ViewModels (for example CreateOrder, AdminOrders, Money) can be split into services to improve maintainability.
- Some entities use string status/category fields; consider enums/value objects to reduce typo risk.
- The app is local-first (SQLite in user profile). Multi-user scenarios require synchronization or server-backed persistence.

11) Conclusion

EliteRestaurant is a feature-rich WPF operations platform with dense business logic around orders, staffing, inventory, and financial tracking. The project already demonstrates coherent MVVM separation and practical local persistence. The strongest next steps are security hardening, test coverage, and modularization of large feature ViewModels.